

Swing or Take? Predicting MLB Batter Decisions

Kyle Duong

Halicioğlu Data Science Institute, UC San Diego

La Jolla, California, USA

[kyduong@ucsd.edu]

Abstract

In baseball, one of the most important decisions a hitter has to make is whether to swing or take a pitch in the split of a second. Chase rate is an important performance metric that measures a batter's plate discipline and is influenced by players' split-second decisions. In this study, I analyzed a sample of 75,000 pitches from the 2025 MLB season using Statcast, MLB's advanced tracking system that provides explicit data on pitch and player performance. I treated the swing decision as a binary classification problem. I was interested in whether pitch location, pitch type, velocity, and matchup context could predict whether a hitter would swing. Through exploratory data analysis, I found that hitters swung at 68% of pitches in the strike zone compared to 30% of pitches outside the strike zone. The swing rate also increased clearly when there were two strikes. Features were also engineered based on the strike zone, count, pitch sequence, and player tendencies to help build our model. I compared several models, which included a majority-class baseline, Naive Bayes, logistic regression, a linear SVM, a decision tree, a random forest, and gradient-boosted trees. The gradient-boosted model performed best of all the models tested, with an accuracy of 0.778 and a ROC-AUC score of 0.863. A stronger model is efficient and good for being accurate and predictive, but engineered features are essential in enhancing the model because, without them, the model is weaker. Adding the engineered features improved AUC by 0.286, while gradient boosting helped to improve AUC further by 0.045. The ablation study showed that pitch location was the most important feature because removing it dropped AUC by 0.22. The results indicated that the effect of feature engineering is greater than the model selection on the performance, compared to the choice of model. Adding the engineered features increased the AUC by 0.286, while switching to gradient boosting provided a smaller improvement of 0.045. The ablation study also showed that pitch location was the most important predictor, as removing it caused the AUC to drop by 0.22. These findings support existing research about players' plate discipline, how much a batter is willing to swing a pitch that is inside or outside the strike zone; the results showcase that pitch location is the most prominent factor in swing decision, with pitch type and velocity also being essential variables in further improving a model.

1 Introduction

In baseball, every pitch forces the batter to make a quick decision. Do you want to swing or do you want to take? Even though it happens in a split second, this decision is one of the most important parts of hitting. Good hitters will swing at pitches that they can handle and avoid chasing pitches that are way outside of the strike zone. This helps them be disciplined, leading to rewards such as drawing walks, avoiding weak contact, and being on base more often. This makes studying the swing/take decision very important to

sport analytics. It connects directly to modern plate discipline stats such as chase rate, in-zone swing rate, and the MLB's Swing/Take Run value. This model, which is designed to predict when hitters swing or take, can help us better understand hitter decision-making, player development, and developing strategies such as pitch-hitting strategies.

The swing decision also works well as a data-mining problem. The outcome is straightforward, as it is either that the batter will swing at the pitch or the batter will take the pitch instead. At the same time, the data includes many useful predictors, such as pitch location, pitch speed, pitch type, count, and game context. These features give the problem enough structure to model, but it is not so obvious that the answer is trivial.

The dataset has over 75,000 pitches, and the target is a binary decision. This allows for the project to be a good fit to use supervised learning methods that were covered during the course. It also gives us a chance to see how simple models compare against more flexible models and see how much feature engineering improves prediction.

Research problem. *Can pitch location, pitch type, velocity, pitch count, and matchup context predict whether an MLB batter swings or takes a pitch?* Answering this question allows us to build a swing-probability model that estimates how likely a hitter is to swing at a given pitch. The model provides a useful reference point for understanding and studying chase rate and swing-decision metrics, while also supporting being able to use of this type of data in real life, such as pitch recommendation, player development, and ways to approach coaching, diving into the world of data analytics.

Hypothesis. Batters are more likely to swing at pitches near or inside the strike zone, especially with two strikes. Pitch type, pitch speed, pitch count, and batter/pitcher handedness should also carry predictive signals for the batter's decision to swing or not.

2 Dataset

2.1 Identifying the Dataset

The dataset is pulled from the MLB 2025 Season Statcast of 75,000 pitches retrieved through the **pybaseball** Python package. For some background, Statcast is MLB's visual and radar tracking system, which records the full physical trajectory of every pitch thrown. It records point & speed, movement, spin, and the location at which the pitch crosses home plate, along with game context. Game context includes the count, the number of outs, the inning, and the score, as well as the categorical outcome of each pitch. Each row is a pitch, and the data spans the entire 2025 regular season from March to September. The dataset has 118 columns. The dataset will need to be cleaned, any leakage has to be removed, and then, after building the swing/take outcome, the final dataset will become a useful item for the project on its own.

2.2 Defining the Target

As defined in the dataset introduction, the data does not store an explicit swing flag, but the description field records the result of every single pitch, from which the batter's intent is explicit. Many outcomes can occur, but we can easily map these descriptions to a binary label. Outcomes that require the bat to move are the following: `swinging_strike`, `foul`, `hit_into_play`, `foul_tip`, and `bunt_attempt` variants will be labeled as a **swing (1)**. Outcomes where the batter lets the pitch pass them, such as `ball`, `called_strike`, `blocked_ball`, and `hit_by_pitch`, are labeled as **take (0)**. There are also additional labels that do not belong in either category: `automatic_ball`, `automatic_strike`, and `pitchout`. These are dropped from the dataset because they are pitch-clock or intentional events that do not reflect **genuine swing decisions**. This cleaning step is motivated by the task itself. It is because we want to model the swing decision, so non-decisions are removed rather than arbitrarily assigned a class. After filtering, 74,739 pitches remain (261 dropped), with an overall swing rate of 0.478. This shows that it is a nearly balanced problem in which accuracy is a meaningful, if insufficient, metric.

2.3 Missingness and Cleaning

A small fraction of pitches are missing physical tracking columns because of tracking dropouts. To be more specific, it just means that it occurs when the camera or radar system fails to capture part of a pitch's path, leaving some of the recorded measurements blank. For the main features, we require the core tracking and strike-zone fields to be present. These variables include the following: `plate_x`, `plate_z`, `sz_top`, `sz_bot`, `release_speed`, `pitch_type`, `zone`, `count`, and `handedness`. Rows missing any of these required fields are dropped from the dataset. For the secondary movement and spin features (`pfx_x`, `pfx_z`, `release_spin_rate`, and `effective_speed`), I decided to use median imputation for this. This was a reasonable choice because a small number of tracking errors produced unusually large values, making the median more stable than the mean.

2.4 Feature Engineering

The raw tracking variables are useful, but they are not always in the most predictive form. To make the data more informative, I decided to engineer features that describe both the geometry of the pitch location and the context of the at-bat.

Strike-zone geometry. The strike zone is roughly 17 inches wide, which is about 0.83 feet from the center to each horizontal edge, once the radius of the baseball is included. Vertically, the zone is defined by the batter-specific top and bottom of the strike zone, `sz_top` and `sz_bot`. We normalize each pitch location so that -1 and 1 represent the edges of the strike zone in each direction. I then compute a single radial distance from the center of the zone:

$$norm_x = \frac{plate_x}{0.83}, \quad norm_z = \frac{plate_z - sz_mid}{sz_halfh}, \quad (1)$$

$$radial_dist = \sqrt{norm_x^2 + norm_z^2}, \quad (2)$$

where `radial_dist` is 0 at the center, 1 at the edge, and greater than 1 outside the zone. As a sanity check to this work, our derived in-zone flag agrees with Statcast's own zone about 96% of the time.

Count context. I added `two_strike`, `three_ball`, `count_diff` ($= balls - strikes$), and flags for whether the hitter or pitcher is ahead in the count. This feature helps capture the pressure of the at-bat.

Pitch type. I grouped the 18 raw pitch codes into four broader pitch types, which include fastball, breaking, offspeed, and other. This makes the feature simpler while still keeping the main differences between pitch types.

Matchup. I add a same-handedness, or platoon, flag along with separate handedness indicators for the batter and pitcher.

Sequence features. This was an important feature to engineer. This is because, within an at-bat, the previous pitch can affect what the batter expects next. After sorting by game, at-bat, and pitch number, I add `prev_swing`, `prev_in_zone`, and `is_first_pitch`. I want to note that these features use only information from before the current pitch, so they do not introduce label leakage.

2.5 Exploratory Data Analysis

To help test my hypothesis and guide the model design, exploratory data analysis will be used.

The strike-zone swing surface. Figure 1 illustrates the swing rate as a function of plate location. The surface is a bright bullseye over the strike zone that fades rapidly outside the zone, and swing rate decreases monotonically with radial distance, crossing 50% almost exactly at the zone edge. The in-zone swing rate is 0.680 versus 0.301 out of zone. This strongly verifies our central hypothesis and shows that the decision boundary is radial and non-linear, which is an indication that flexible tree-based models will outperform pure linear ones.

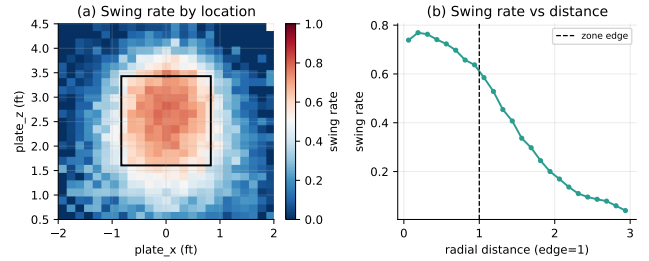


Figure 1: (a) Empirical swing rate over plate location (catcher's view) with the average strike zone outlined. (b) Swing rate decreases monotonically with radial distance from the zone center, crossing 50% at the zone edge.

Count effects. Figure 2 shows the swing rate by count. The swing rate increases with strikes as hitters protect the plate, and the characteristic 3-0 take (swing rate ≈ 0.10) and aggressive 3-2 count (≈ 0.71) are clearly visible. Location effect is strongly modulated by count, i.e., there is an *interaction* between location and count, which is another argument for a model that captures interaction.

Pitch type, velocity, handedness. As shown in Figure 3, off-speed pitches also induce slightly more swings than breaking balls (Figure 3), and the release-speed distributions of swings and takes overlap quite a bit. The raw platoon effect on swing rate is modest. These are secondary and context-dependent cues, which are consistent with our hypothesis that they refine but do not define the decision.

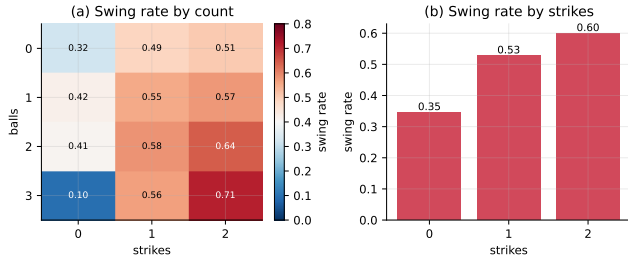


Figure 2: (a) Swing rate by ball–strike count, showing the 3-0 take and 3-2 aggression are clearly visible. (b) Swing rate rises steadily with the number of strikes.

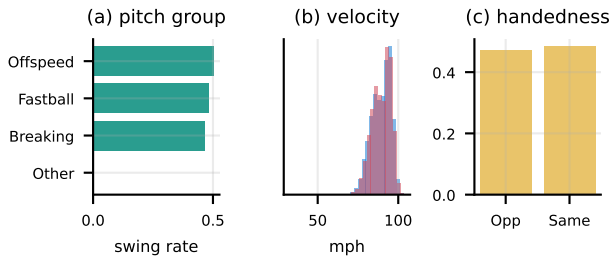


Figure 3: Secondary cues. (a) Swing rate by pitch group. (b) Release-speed distributions for swings versus takes overlap heavily. (c) Swing rate by handedness matchup. None of these dominates the decision the way location does.

Feature correlations. Figure 4 shows the Pearson correlations of the main features with the target. The swing label has the largest correlation with the in-zone flag (positive) and radial distance (negative). The predictors are mostly uncorrelated with each other, besides the expected clustering among the location features. This near-independence is why Naive Bayes is a defensible baseline, but still misspecified: its conditional independence assumption is roughly valid everywhere but the location group.

Unsupervised structure. Figure 5 reports a PCA scree plot and a K-Means clustering of normalized plate location using the dimensionality-reduction and clustering methods from lecture [14]. PCA shows that three to four components capture most of the variance in the physical pitch descriptors, so heavy dimensionality reduction is unnecessary and I kept the interpretable raw features. K-Means ($k = 6$) recovers an intuitive geography: the central cluster has a 0.706 swing rate while the peripheral clusters sit near 0.30–0.49, confirming that the decision is highly separable in location space.

Association rules. Treating each pitch as a transaction of boolean items, we mine simple $\{\text{context}\} \rightarrow \{\text{decision}\}$ rules. The strongest positive rule is $\text{in_zone} \rightarrow \text{swing}$ (confidence 0.68, lift 1.42), followed by $\text{out_zone} \rightarrow \text{take}$ (confidence 0.70, lift 1.34) and $\text{two_strike} \rightarrow \text{swing}$ (confidence 0.60, lift 1.26). Every rule’s direction matches the hypothesis, and location and count carry the bulk of the deterministic signal.

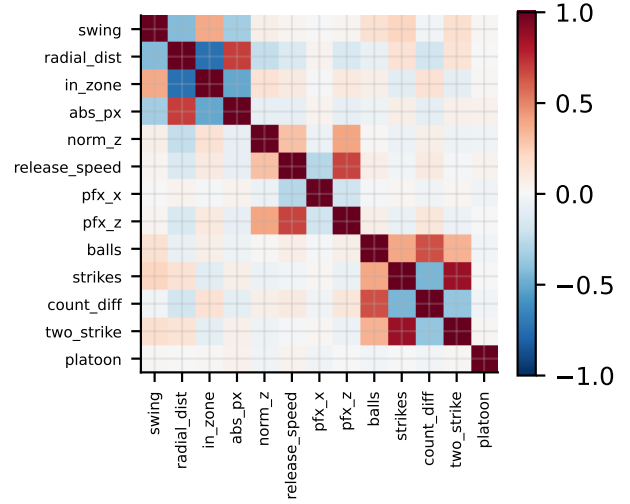


Figure 4: Pearson correlation among features and the swing target. Location features carry the strongest relationship with swinging, while most other predictors are weakly correlated.

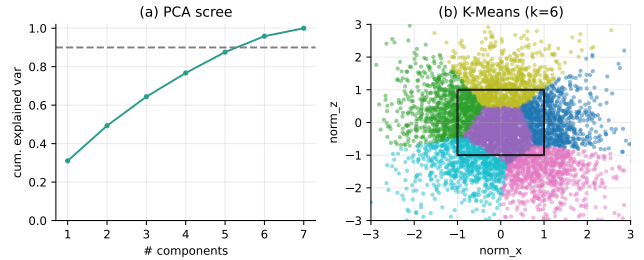


Figure 5: (a) PCA scree plot of physical pitch descriptors: a few components capture most variance. (b) K-Means clusters over normalized location, where it shows the central cluster has the highest swing rate.

These EDA results motivate the model design in conjunction. Location is everything, and the effect is radial. Count modulates location, so interactions are important. Pitch Type, Velocity, and Handedness help to give some context. The features are mostly independent, except for the location group, which makes Naive Bayes a defensible but misspecified baseline.

3 Predictive Task

Task. I predict, for each pitch, whether the batter swung (1) or took (0) using only information available at the moment the pitch crosses the plate: location, velocity, movement, pitch type, count, outs, inning, matchup, and within-at-bat history.

Evaluation. Accuracy is meaningful because the swing and take classes are almost balanced. But accuracy alone is not enough. I also proposed implementing a few additional metrics to evaluate the models more comprehensively. F1-score is used to check that the

model is performing well on both classes, not only the majority class. The main metric is ROC-AUC as this is a probabilistic swing model. I also reported PR-AUC, i.e., average precision, and log-loss to assess the quality of the predicted probabilities. This is significant because the predicted probabilities are used in downstream applications like swing-probability surfaces and chase-rate analysis, not just the final swing/take label.

Validation. I utilize an 80/20 stratified train-test split, keeping the swing label for stratification. This yields 59,006 training pitches and 14,752 test pitches, both maintaining the same swing rate. Given that swings on separate pitches are nearly independent, a random separation suffices for measuring the model's ability to deal with new pitches for known players. There are some restrictions on the sleek encodings for batters and pitchers whose encodings are smoothed. The encodings are fit on the training set only. Therefore, they are applied to the test set to prevent leakage.

Baselines and motivation. I establish several benchmarks for my comparisons, and each benchmark tests a different modeling assumption. The *majority-class* classifier is the simplest baseline model because it always predicts the most common class. It establishes a minimal baseline for the other, more informative models. *Gaussian Naive Bayes* attempts to model situations where the features of observations are largely independent. Some exploratory data analysis (EDA) reveals that several features, particularly location features, are not completely independent and are somewhat correlated. Therefore, Gaussian Naive Bayes' poorer performance helps show the importance of modeling relationships and interactions between features. *Logistic regression*, with L2 regularization following the overfitting and regularization material from lecture [13], is the main baseline model for the family of linear models. Its simplicity and interpretability make it an important benchmark. However, linear models lack the ability to capture complex, multi-dimensional relationships, such as the rounded boundaries of the strike zone or the location $\times$ count interaction. I also considered a *linear support vector machine (SVM)* to determine if this is a general limitation of linear models. To demonstrate the importance of ensemble methods such as random forests and gradient boosting, I also implemented one unpruned *decision tree*, which acts as a high-variance non-linear learner.

Why I expect the model to win. The final model I used was a *gradient-boosted tree ensemble*, which connects to the bagging and boosting material from lecture [12]. This technique builds multiple small decision trees sequentially, with each successive tree working to address the errors of its predecessor. This model works well for the swing/take problem because the problem is not always solved in a linear manner. For example, hitters will respond differently to a pitch that is almost directly in the center of the strike zone compared to a pitch that is near the edge or almost completely out of the strike zone. This model also captures interactions between features, such as pitch location and pitch count. Due to this, I expect gradient boosting to outperform the linear baselines in terms of AUC. It should also outperform a solitary decision tree model because the ensemble reduces the variance that comes from relying on only one tree.

4 Model

4.1 Feature Matrix and Models

I created a final design matrix for the models that brings together all the features I have engineered. I one-hot encoded *pitch_group* since it is the only categorical feature. The other features remain unaltered since they are numeric. To eliminate data leakage, I excluded features that reveal the outcomes post-pitch. Examples of these features are *description*, *events*, *launch_speed*, and *bat_speed*. For Naive Bayes, logistic regression, and SVM, I standardized features due to their sensitivity to feature scaling. I used the original feature values for tree-based models since this step is unnecessary because tree-based models are scale-invariant. I primarily used scikit-learn's [9] *HistGradientBoostingClassifier*. This implementation is a faster histogram-based version of gradient boosting, in the same general algorithmic family as XGBoost and LightGBM, where many decision trees are built and combined. My specifications for this are 400 iterations, a learning rate of 0.08, 63 leaves, and an L2 regularization value of 1.0.

4.2 Feature Design and Failed Experiments

The most notable engineered feature is radial distance. Among all the pitches, radial distance simplifies the measure of how far away a pitch is from the center of the strike zone. This is consistent with a generalization of baseball since hitters are more likely to swing at a pitch that is closer to the middle of the strike zone, and further away from the edges of the strike zone. Normalizing the strike zone for each batter is crucial since not every batter maintains the same strike zone. These features are helpful combined since they give insight into the batter and pitcher swing involvement while maintaining a limit to the model's leakage.

Some experiments were unhelpful and thus were omitted. Using all eighteen pitch types caused moderate overfitting and increased complexity, while not impacting AUC. Overfitting was avoided by simplifying the categories into four primary categories. These interactions were polynomial terms that increased the model's complexity. I thus allowed the tree-based model to learn these interactions. Spin rate and spin axis also offered little help in predicting Swing/No-Swing. This isn't a big surprise since spin relates to the trajectory and behavior of a pitch, both pre- and post-contact with the bat. It isn't inherently tied to the batter's decision to swing. I also removed the RBF-kernel SVM because of how slowly it ran on this dataset. This is because kernel SVMs run much slower as the training dataset size increases, meaning it didn't finish in a reasonable timeframe with the 59,006 training pitches. For this reason, I took the simplest approach and used a linear kernel SVM as a baseline and histogram-based gradient boosting as the model.

5 Literature

The swing/take decision is located in a well-established literature involving plate-discipline metrics, machine-learning applications, and recent academic modeling. I examined the most analogous literature and compared it to our methods.

Data Tracking. PITCHf/x started public pitch tracking around 2006 [4], and later tracking systems came about in 2020 when MLB's Statcast/Hawk-Eye system was introduced. I accessed this

data through the open-source pybaseball package [5], which is the same acquisition tool used by much of the applied community and the work from Penn State surveyed below.

Plate-discipline metrics. Our task involves analyzing two typical plate-discipline statistics from FanGraphs: O-Swing%, which indicates how often a batter swings at a pitch outside the strike zone, and Z-Swing%, which shows how often a batter swings at a pitch that is within the strike zone [2, 3]. These statistics correspond exactly to the conditional swing rates that our association-rule analysis captures. The parallelism is remarkable: FanGraphs indicates that the league average Z-Swing% and O-Swing% is approximately 67% and 30%, respectively, while our in-zone and out-of-zone swing rates are 0.680 and 0.301, respectively. These statistics also strengthen our strike-zone modeling, as they correspond to a well-documented baseball statistic. An additional example would be the Swing/Take Run Value metric from MLB’s Baseball Savant, which divides the strike zone into attack zones and assigns a value to each outcome [8]. A metric is reliant on the knowledge of the expected swing behavior, which is what our calibrated swing-probability model does.

Most similar applied work. The most similar work to ours appears to be Wilson’s [15]. He created a LightGBM [6] gradient-boosted swing-probability model built on Statcast data using essentially the same features as us: location of the pitch, its speed, vertical and horizontal break, the count, the previous pitch, etc. He achieved an accuracy of 79.5%, which then improved to an accuracy of 80.5% after some parameter tuning. Like us, Wilson believed location is “definitely the most important factor,” which our permutation-importance and ablation tests support, as location has the highest impact and removing it has a cost of 0.22 AUC. Our work also has three main differences. Wilson built a per-hitter model and used data from Joey Votto for five seasons of his career, while I built a single population model for all hitters and added personalization by using leakage-safe batter/pitcher target-encoded tendencies, which is a method of personalization for hitters and pitchers with very limited data. Wilson also only measured accuracy, while I also measured other metrics, including ROC-AUC, PR-AUC, log-loss, and a calibration test, as swing probability actually relies on probabilities and not the prediction itself. Lastly, controlled ablation and feature versus model tests were run, which Wilson did not perform.

Academic modeling. Powers and Yurko’s model simulates a batter’s decision to swing based on the count and pitch location using a Bayesian hierarchical model [10]. This is the same conditioning structure as our EDA, which is a location and count interaction. However, they are taking a different approach because they are aiming to interpret bat-tracking metrics, while I look to classify swing versus take. Their more hierarchical and batter-specific approach is a more formal version of our target-encoding and player tendencies, and it serves as a good direction for the rest of our work. In a more general sense, Koseler and Stephan give a broad overview of how baseball and machine learning intersect [7]. They note how game-state variables, which they refer to as context variables, usually include the count, base occupancy, number of outs, and score. These variables are used when constructing pitch-level models, which agrees with our count and context variables.

Modern Innovation & Our Stance. To the best of my knowledge, the world model from Ahn et al. [1] is the most impressive

recent model I was aware of. They consider games as sequences of events and pre-train on over 7 million pitch sequences. They report an accuracy of 78% on the batter’s swing decision. Our gradient boosted model has an accuracy of 77.8%. This is essentially the same as the accuracy reported by the LLM and was achieved using a dataset of 75,000 pitches, a model that trains in seconds as opposed to on the scale of billions of tokens, and fully interpretable features. This is a generalizable observation in the literature. When working on tabular pitch data, simpler tree ensembles can outperform large neural models and can be more cost-effective and easier to interpret. Rather than contributing new tasks, I tackled this model from a clean course-method approach. To design the classical location-and-count findings in the frameworks of association-rule and clustering them from a PCA viewpoint, they must first be cross-checked with thorough exploratory data analysis. I have also included a light within-at-bat sequence feature, a series of controlled ablation, calibration, robustness experiments, and feature/model decomposition, which differ from what most blogs and preexisting academic work do not provide. Ultimately, the results align with pre-existing literature and research. The location and count features are dominant. The gradient-boosting model is competitive with the best models I have documented. I also clearly stated and quantified each of the model gaps we identified.

6 Results

6.1 On Why, Not Just What

High accuracy alone is not a sufficient metric for our model. Therefore, I aimed to provide an interpretation of the performance for each of the results listed. We designed a series of experiments, where I varied one dependent factor, to assess the results. A simple-feature baseline allows us to quantify the contributions from the model and the features themselves. An ablation study lets us measure the contribution of each feature group to AUC, and a sensitivity analysis shows that the results aren’t caused by the arbitrary selection of a specific hyperparameter value.

6.2 Separating Feature Value from Model Value

Before I compare the models, I start with a basic model: logistic regression with only the basic pitch location features. Table 1 looks at this basic location-only model in contrast to logistic regression with the designed features and to the final gradient-boosted model. The results show that the designed features dominate the rest. From the basic location features to the designed features, the AUC improves by +0.286. I then change from logistic regression to gradient boosting, following the ensemble learning ideas from lecture [12], with those features, and the AUC improves by +0.045. From this, I conclude that the biggest change to the model was from the designed features. The gradient-boosted model improves the outcome of the model, but the change was smaller since it mainly allows the model to capture interactions between the features.

6.3 Model Comparison

Table 2 summarizes performance on the held-out test set, while Figure 6 shows the ROC curves and confusion matrix of the best model. The results align with the expectations based on the exploratory analysis. Naive Bayes performs just above the baseline,

Table 1: Decomposing total lift into feature and model contributions.

Configuration	Accuracy	ROC-AUC
Logistic regression, location only	0.470	0.532
Logistic regression, all features	0.737	0.818
HistGradientBoosting, all features	0.778	0.863
Feature-engineering lift	—	+0.286
Model lift	—	+0.045

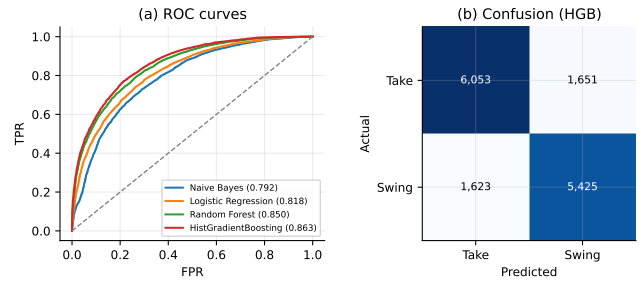
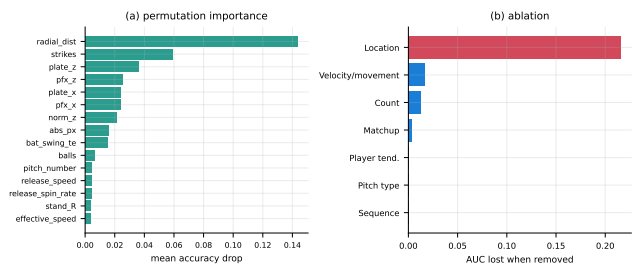
Table 2: Test-set performance of all models. The proposed gradient-boosted model is best on every metric.

Model	Acc.	F1	ROC	PR	LogL
Majority	0.522	0.000	0.500	0.478	17.22
Naive Bayes	0.551	0.678	0.792	0.755	4.07
Logistic Regression	0.737	0.726	0.818	0.797	0.521
Linear SVM	0.738	0.727	0.818	0.797	—
Decision Tree	0.737	0.724	0.820	0.801	0.553
Random Forest	0.763	0.750	0.850	0.839	0.478
HistGradBoost	0.778	0.768	0.863	0.852	0.458

and its high log-loss shows that its predicted probabilities cannot be trusted. This intuitively makes sense since Naive Bayes assumes that all features are independent. However, important pitch location features are not independent. The performance of the linear models is better, with an accuracy of about 0.74 and an AUC of about 0.82. These results show that a linear model can capture most of the main structure in the data. However, linear models are inherently limited because they cannot fully capture non-linear boundaries or feature interactions. Random forests are a collection of decision trees and reduce the variance limitations of a single decision tree through bagging. The best model is HistGradientBoosting. The jump from linear models to boosting models is about a 4 percentage point improvement in accuracy, with an AUC improvement of about 0.05. These results show that swing decisions depend on non-linear patterns and interactions between features. The confusion matrix shows that the results are balanced, as precision and recall are about 0.79 and 0.77 for both swing and take. This means the model is not simply favoring one class over the other.

6.4 Feature Importance

Figure 7a shows permutation importance on the test set. In permutation importance, each feature is shuffled one at a time, and the drop in accuracy is measured. Large drops in accuracy indicate that the feature is important. The first conclusion that can be drawn is that location features are the most important predictors. Shuffling *radial_dist* is the most impactful, since accuracy drops by 0.143. After that, the ordering of importance is *strikes*, the other pitch location features, and the player-tendency encodings. Individually, pitch type, pitch velocity, pitch spin, and pitcher handedness are ranked as less important features. This does not mean that they are not useful. Permutation importance reviews one feature at a time

**Figure 6: (a) ROC curves for four representative models. (b) Confusion matrix of the gradient-boosted model on the test set.****Figure 7: (a) Permutation importance (top 15 features) for the gradient-boosted model. (b) Ablation study: AUC lost when each feature group is removed. Both views agree that location dominates.**

while the others are still included, so the ablation study below tests whether these lower-ranked features still contribute when they are removed as a group.

6.5 Ablation Study

I also performed an ablation study where I omitted one feature group at a time and then trained the final model again. The decrease in AUC represents the marginal value of each feature group. As shown in Figure 7b, the omission of the *location* group is the most damaging. The AUC drops from 0.863 to 0.648, for an approximate 0.22 loss, confirming location as the backbone of the swing decision. Next in significance are the *velocity/movement* and *count* groups. The player-tendency, pitch type, pitch sequence, and matchup groups each contribute a positive but smaller incremental effect. These groups help refine the decision boundary in ambiguous regions, but they do not define the main pattern. From this analysis, the engineered feature groups are beneficial, but location is the overwhelmingly dominant group.

6.6 Robustness and Calibration

To ensure that one train-test split is not the cause of the primary finding, I examined the in-zone and out-of-zone swing rates across different partitions of the data. As shown in Figure 8, panel (a), the size of the gap is remarkably consistent, about 0.38, across left- and right-handed batters, left- and right-handed pitchers, and

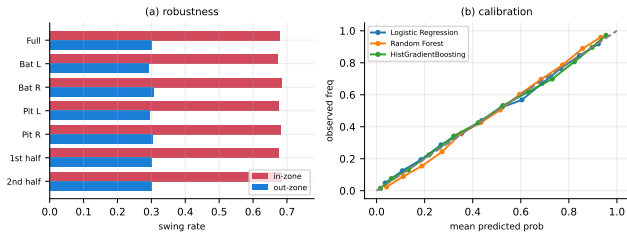


Figure 8: (a) The in-zone vs. out-of-zone swing gap is stable (≈ 0.38) across-handedness and season-half subsamples. (b) Calibration curve: the gradient-boosted model’s probabilities are well-calibrated.

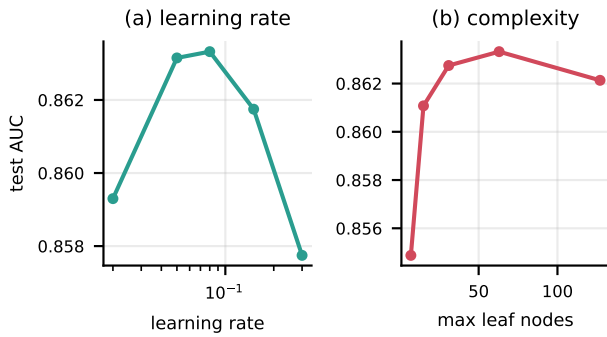


Figure 9: Hyperparameter sensitivity of the gradient-boosted model. Test AUC is stable across (a) learning rate and (b) tree complexity, varying by only a few thousandths, indicating the result is robust to tuning.

both halves of the season. This indicates that the main pattern is consistent across different subsets of the data. Figure 8, panel (b), shows that the boosted model is well-calibrated because the predicted probabilities track the diagonal closely. For example, if the model predicts a swing probability of 0.7, then the observed swing rate is roughly 70%. This validates using the model’s predicted probabilities directly as a swing-probability surface.

6.7 Hyperparameter Sensitivity

I tested whether the boosted model is sensitive to its hyperparameters by sweeping the learning rate and tree complexity. As shown in Figure 9, test AUC stays within a narrow range, from 0.857 to 0.863, across the values tested. This shows that the model is not dependent on one exact setting and that the reported improvement is robust. A very high learning rate and very deep trees slightly lower AUC, which indicates a bit of overfitting. This is in line with the regularization concepts we discussed in class, so I select hyperparameter values from the stable middle area of the surface.

6.8 Case Study: Where the Model Succeeds and Fails

Through examining the model’s best educated guesses, I was able to better relate aggregation results to one’s common knowledge

Table 3: Representative case-study predictions. Confident correct calls are easy in-zone two-strike pitches. Confident errors are out-of-zone chases or disciplined takes of hittable pitches.

Type	radial	count	group	p_{swing}	actual
Correct	0.75	1-2	Fastball	0.987	swing
Correct	0.88	2-2	Fastball	0.986	swing
Error	3.85	2-2	Breaking	0.001	swing
Error	2.52	0-2	Fastball	0.008	swing
Error	0.14	1-2	Fastball	0.982	take

about hitting in baseball. I examined the model’s most confident correct predictions and its most confident mistakes, as shown in Table 3. The most confident correct predictions by the model are all situations where any hitter would agree: in-zone fastballs in a two-strike count. These are the low-hanging fruit, as they have high-confidence swing probabilities near 0.99 and corresponding swings by the batters.

The model’s most confident errors occur in two different ways. The first is an easily identifiable situation: a pitch well outside the zone, with radial distance between 2 and 4, during a two-strike count. The model predicts that this pitch will almost always be taken, with $p_{\text{swing}} \approx 0.01$, but instead the batter swings. This represents a defensive-minded approach, where the hitter is trying to protect the plate and avoid taking strike three. Since there is no additional information available to the model regarding why the batter swung at the pitch, it cannot capture the “why” behind the swing.

The second type of confident error occurs when the model predicts a swing on a center-cut pitch, with radial distance 0.14 and $p_{\text{swing}} \approx 0.98$, but the disciplined hitter takes the pitch intentionally to continue working the count. In either case, the model is unable to account for some degree of hitter intent that is not captured using location and count data.

6.9 Limitations

This analysis has several limitations that were noted throughout the process of building this model. The first is that I used a single sample of 75,000 pitches from only one MLB season. Although the main result holds across the data’s various sections, the conclusion would be stronger with more season and league model checks. The second limitation is that many remaining prediction errors come from tough cases. This is especially common for borderline pitches with radial distance near 1, where the swing decision is genuinely ambiguous. Some prediction errors also come from factors the model cannot fully observe, such as the batter’s approach, scouting reports, game preparation, intent, or other unknowns. A model based mainly on location and count cannot fully capture these factors. The third limitation is that the model’s player-specific component is limited. I only used smoothed swing-rate encodings for batters and pitchers. This could be improved with more advanced player-specific models, such as latent-factor models, as well as richer pitch information, including full pitch trajectories or pitch tunneling features. Lastly, I use a random train-test split instead

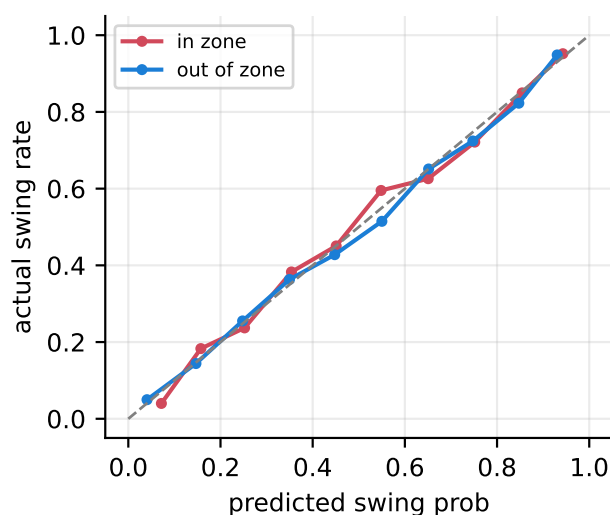


Figure 10: Actual swing rate versus predicted probability, split by zone. Both curves follow the diagonal at the extremes; errors concentrate in the ambiguous 0.3–0.7 band.

of a time-based split. A strictly chronological split would better assess how the model performs in a real deployment setting, where it must predict future pitches using only past data.

6.10 Major Takeaways

Swing/take decisions can be predicted fairly well by pitch location, count, and matchup context, among other elements. The gradient-boosted model, with the best calibrated probabilities, achieves an impressive 0.778 and 0.863 for accuracy and ROC-AUC, respectively. The hypothesis was proven correct and calculated: hitters tend to swing at pitches in or near the strike zone and generally swing at more pitches as the count nears two strikes. Pitch type, pitch speed, and pitch handedness add some value to the context, but are considered secondary indicators. When looking purely at performance, the order is boosting > bagging > single tree $\approx$ linear > Naive Bayes. Coupling this with model and feature enhancement comparison, it quickly becomes obvious that feature engineering is of paramount importance. Balancing model accuracy, interpretability, and efficiency, the model is advanced enough to consider practical use cases in baseball for things like pitch recommendation tools, player development, and swing probability overlays.

Acknowledgments

I thank Professor Jingbo Shang and the DSC 148 teaching staff for their instruction and guidance throughout the course [11].

References

- [1] Young Jin Ahn, Yiyang Du, Zheyuan Zhang, and Haisen Kang. 2026. Neural Sabermetrics with World Model: Play-by-play Predictive Modeling with Large Language Model. *arXiv preprint arXiv:2602.07030* (2026). <https://arxiv.org/abs/2602.07030>
- [2] David Appelman. 2008. Plate Discipline Stats. FanGraphs. <https://blogs.fangraphs.com/plate-discipline-stats/> Accessed 2026.
- [3] FanGraphs. [n. d.]. Plate Discipline (O-Swing%, Z-Swing%, etc.). FanGraphs Sabermetrics Library. <https://library.fangraphs.com/offense/plate-discipline/> Accessed 2026.
- [4] Mike Fast. 2010. What the Heck is PITCHf/x? *The Hardball Times Baseball Annual* (2010), 153–158.
- [5] James James and pybaseball contributors. 2017. pybaseball: A Python Package for Baseball Data Acquisition and Analysis. GitHub. <https://github.com/jldbc/pybaseball>
- [6] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30.
- [7] Kaan Koseler and Matthew Stephan. 2017. Machine Learning Applications in Baseball: A Systematic Literature Review. *Applied Artificial Intelligence* 31, 9–10 (2017), 745–763.
- [8] MLB Advanced Media (Baseball Savant). [n. d.]. Statcast Swing/Take Run Value Leaderboard. baseballsavant.mlb.com/leaderboard/swing-take Accessed 2026.
- [9] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, et al. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
- [10] Scott Powers and Ronald Yurko. 2025. Swinging, Fast and Slow: Interpreting Variation in Baseball Swing Tracking Metrics. *arXiv preprint arXiv:2507.01238* (2025). <https://arxiv.org/abs/2507.01238>
- [11] Jingbo Shang. 2026. DSC 148: Introduction to Data Mining. Course lectures, Halicioğlu Data Science Institute and Department of Computer Science and Engineering, University of California San Diego. Spring 2026.
- [12] Jingbo Shang. 2026. DSC 148 Lecture: Ensemble Learning — Bagging and Boosting. Course lecture, University of California San Diego. Spring 2026.
- [13] Jingbo Shang. 2026. DSC 148 Lecture: Overfitting and Regularization. Course lecture, University of California San Diego. Spring 2026.
- [14] Jingbo Shang. 2026. DSC 148 Lectures: K-Means Clustering and Principal Component Analysis. Course lectures, University of California San Diego. Spring 2026.
- [15] Jeremiah Wilson. 2020. Modeling Swing Probability: Applying Machine Learning to Baseball. Towards Data Science. <https://towardsdatascience.com/modeling-swing-probability-b02b5ab7fbb5/> Accessed 2026.